

Quick Start Guide

Prerequisites

- Python 3.8+
- CUDA-capable GPU (optional, but recommended)
- 10GB+ free disk space for dataset

Installation (5 minutes)

1. Clone the Repository

```
git clone https://github.com/yourusername/industrial-defect-detection.git
cd industrial-defect-detection
```

2. Create Virtual Environment

```
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate
```

3. Install Dependencies

```
pip install -r requirements.txt
```

Download Dataset (15-30 minutes)

Option 1: Using the Script (Recommended)

```
bash scripts/download_data.sh
```

Option 2: Manual Download

1. Visit [MVTec AD Dataset](https://www.mvtec.com/company/research/datasets/mvtec-ad) (<https://www.mvtec.com/company/research/datasets/mvtec-ad>)
2. Download `mvtec_anomaly_detection.tar.xz` (~4.3 GB)
3. Extract to `data/raw/`

```
mkdir -p data/raw
tar -xvf mvtec_anomaly_detection.tar.xz -C data/raw/
```

Training Your First Model (1-2 hours)

Quick Test (Small Category)

```
# Train on 'bottle' category (smallest, fastest)
python train.py --config configs/config.yaml --category bottle --epochs 50
```

Full Training

```
# Train on any category with full configuration
python train.py --config configs/config.yaml --category cable --epochs 100
```

Monitor Training

Watch the progress bar and logs:

- Training loss decreases
- Validation Dice coefficient increases
- Checkpoints saved every 10 epochs

Results saved to:

- `results/checkpoints/` - Model checkpoints
- `results/logs/` - Training logs and metrics
- `results/visualizations/` - Sample predictions

Running Inference (< 1 minute)

On Test Images

```
python inference.py \
  --checkpoint results/checkpoints/best_model.pth \
  --image data/raw/mvtec_ad/bottle/test/broken_large/000.png \
  --output prediction_result.png
```

Batch Inference

```
from src.models import UNet
from src.utils.helpers import load_checkpoint
import torch

model = UNet(n_channels=3, n_classes=1)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
load_checkpoint(model, 'results/checkpoints/best_model.pth', device=device)

# Run inference on your images
# ... (see inference.py for details)
```

Exploring the Notebook (5 minutes)

```
jupyter lab notebooks/demo.ipynb
```

The notebook includes:

- Dataset exploration
- Model architecture visualization
- Training metrics analysis
- Inference examples

Project Structure Overview

```
industrial-defect-detection/  
├── src/                # Source code  
│   ├── models/        # U-Net architectures  
│   ├── data/          # Dataset loading  
│   └── utils/         # Utilities  
├── configs/           # Configuration files  
├── scripts/           # Automation scripts  
├── notebooks/         # Jupyter notebooks  
├── tests/             # Unit tests  
├── train.py           # Training script  
└── inference.py       # Inference script
```

Common Issues & Solutions

Issue: Out of Memory

Solution: Reduce batch size in `configs/config.yaml`

```
training:  
  batch_size: 8 # Reduce from 16
```

Issue: Slow Training

Solution:

1. Use GPU if available
2. Reduce number of workers: `num_workers: 2`
3. Use smaller image size: `image_size: [128, 128]`

Issue: Dataset Not Found

Solution: Check dataset path in `configs/config.yaml`

```
data:  
  root_dir: "data/raw/mvtec_ad" # Verify this path exists
```

Next Steps

1. **Experiment:** Try different categories, hyperparameters
2. **Visualize:** Check `results/visualizations/` for predictions
3. **Optimize:** Tune learning rate, loss weights, augmentation
4. **Deploy:** Export best model for production use

Performance Tips

For Faster Training:

- Use GPU (CUDA)
- Increase batch size (if GPU memory allows)
- Use multiple workers for data loading
- Enable mixed precision training (add to code)

For Better Results:

- Train for more epochs (100+)
- Try different architectures (U-Net++)
- Adjust loss function weights
- Use ensemble of models
- Fine-tune hyperparameters

Expected Results

After 50-100 epochs, you should see:

```
Training Metrics:  
- Train Loss: ~0.15  
- Val Loss: ~0.20  
- Val Dice: 0.85-0.90  
- Val IoU: 0.75-0.85
```

Getting Help

- Check `README.md` for detailed documentation
- Review `CONTRIBUTING.md` for development guidelines
- Open GitHub issues for bugs or questions
- Review example notebooks for usage patterns

Customization

Use Different Architecture

Edit `configs/config.yaml` :

```
model:  
  architecture: "unetplusplus" # Change from "unet"
```

Train on Multiple Categories

```
bash scripts/train_all_categories.sh
```

Adjust Augmentation

Edit `src/data/transforms.py` to add/modify augmentation.

Validation

Run tests to ensure everything works:

```
# Run all tests
pytest tests/ -v

# Quick model test
python src/models/unet.py
```

Resources

- [MVTec AD Dataset Paper](https://www.mvtec.com/fileadmin/Redaktion/mvtec.com/company/research/datasets/mvtec_ad.pdf) (https://www.mvtec.com/fileadmin/Redaktion/mvtec.com/company/research/datasets/mvtec_ad.pdf)
- [U-Net Paper](https://arxiv.org/abs/1505.04597) (https://arxiv.org/abs/1505.04597)
- [PyTorch Documentation](https://pytorch.org/docs/) (https://pytorch.org/docs/)

You're ready to start! 🚀

Run your first training with:

```
python train.py --config configs/config.yaml --category bottle --epochs 50
```